

Packet Format

This document describes the MeshCore packet format.

- 0xYY indicates YY in hex notation.
- 0bYY indicates YY in binary notation.
- Bit 0 indicates the bit furthest to the right: 0000000X
- Bit 7 indicates the bit furthest to the left: X0000000

Version 1 Packet Format

This is the protocol level packet structure used in MeshCore firmware v1.12.0

[header][transport_codes(optional)][path_length][path][payload]

- [header](#) - 1 byte
 - 8-bit Format: 0bVPPPPRR - V=Version - P=PayloadType - R=RouteType
 - Bits 0-1 - 2-bits - [Route Type](#)
 - 0x00/0b00 - ROUTE_TYPE_TRANSPORT_FLOOD - Flood Routing + Transport Codes
 - 0x01/0b01 - ROUTE_TYPE_FLOOD - Flood Routing
 - 0x02/0b10 - ROUTE_TYPE_DIRECT - Direct Routing
 - 0x03/0b11 - ROUTE_TYPE_TRANSPORT_DIRECT - Direct Routing + Transport Codes
 - Bits 2-5 - 4-bits - [Payload Type](#)
 - 0x00/0b0000 - PAYLOAD_TYPE_REQ - Request (destination/source hashes + MAC)
 - 0x01/0b0001 - PAYLOAD_TYPE_RESPONSE - Response to REQ or ANON_REQ
 - 0x02/0b0010 - PAYLOAD_TYPE_TXT_MSG - Plain text message
 - 0x03/0b0011 - PAYLOAD_TYPE_ACK - Acknowledgment
 - 0x04/0b0100 - PAYLOAD_TYPE_ADVERT - Node advertisement
 - 0x05/0b0101 - PAYLOAD_TYPE_GRP_TXT - Group text message (unverified)
 - 0x06/0b0110 - PAYLOAD_TYPE_GRP_DATA - Group datagram (unverified)
 - 0x07/0b0111 - PAYLOAD_TYPE_ANON_REQ - Anonymous request
 - 0x08/0b1000 - PAYLOAD_TYPE_PATH - Returned path
 - 0x09/0b1001 - PAYLOAD_TYPE_TRACE - Trace a path, collecting SNR for each hop
 - 0x0A/0b1010 - PAYLOAD_TYPE_MULTIPART - Packet is part of a sequence of packets
 - 0x0B/0b1011 - PAYLOAD_TYPE_CONTROL - Control packet data (unencrypted)
 - 0x0C/0b1100 - reserved
 - 0x0D/0b1101 - reserved

- 0x0E/0b1110 - reserved
- 0x0F/0b1111 - PAYLOAD_TYPE_RAW_CUSTOM - Custom packet (raw bytes, custom encryption)
- Bits 6-7 - 2-bits - [Payload Version](#)
 - 0x00/0b00 - v1 - 1-byte src/dest hashes, 2-byte MAC
 - 0x01/0b01 - v2 - Future version (e.g., 2-byte hashes, 4-byte MAC)
 - 0x02/0b10 - v3 - Future version
 - 0x03/0b11 - v4 - Future version
- transport_codes - 4 bytes (optional)
 - Only present for ROUTE_TYPE_TRANSPORT_FLOOD and ROUTE_TYPE_TRANSPORT_DIRECT
 - transport_code_1 - 2 bytes - uint16_t - calculated from region scope
 - transport_code_2 - 2 bytes - uint16_t - reserved
- path_length - 1 byte - Encoded path metadata
 - Bits 0-5 store path hash count / hop count (0-63)
 - Bits 6-7 store path hash size minus 1
 - 0b00: 1-byte path hashes
 - 0b01: 2-byte path hashes
 - 0b10: 3-byte path hashes
 - 0b11: reserved / unsupported
- path - hop_count * hash_size bytes - Path to use for Direct Routing or flood path tracking
 - Up to a maximum of 64 bytes, defined by MAX_PATH_SIZE
 - Effective byte length is calculated from the encoded hop count and hash size, not taken directly from path_length
 - v1.12.0 firmware and older only handled legacy 1-byte path hashes and dropped packets whose path bytes exceeded [64 bytes](#)
- payload - variable length - Payload Data
 - Up to a maximum 184 bytes, defined by MAX_PACKET_PAYLOAD
 - Generally this is the remainder of the raw packet data
 - The firmware parses this data based on the provided Payload Type
 - v1.12.0 firmware and older drops packets with payload sizes [larger than 184](#)

Packet Format

Field	Size (bytes)	Description
header	1	Contains routing type, payload type, and payload version
transport_codes	4 (optional)	2x 16-bit transport codes (if ROUTE_TYPE_TRANSPORT_*)
path_length	1	Encodes path hash size in bits 6-7 and hop count in bits 0-5
path	up to 64 (MAX_PATH_SIZE)	Stores hop_count * hash_size bytes of path data if applicable
payload	up to 184 (MAX_PACKET_PAYLOAD)	Data for the provided Payload Type

NOTE: see the [Payloads](#) documentation for more information about the content of specific payload types.

Header Format

Bit 0 means the lowest bit (1s place)

Bits	Mask	Field	Description
0-1	0x03	Route Type	Flood, Direct, etc
2-5	0x3C	Payload Type	Request, Response, ACK, etc
6-7	0xC0	Payload Version	Versioning of the payload format

Route Types

Value	Name	Description
0x00	ROUTE_TYPE_TRANSPORT_FLOOD	Flood Routing + Transport Codes
0x01	ROUTE_TYPE_FLOOD	Flood Routing
0x02	ROUTE_TYPE_DIRECT	Direct Routing
0x03	ROUTE_TYPE_TRANSPORT_DIRECT	Direct Routing + Transport Codes

Path Length Encoding

path_length is not a raw byte count. It packs both hash size and hop count:

Bits	Field	Meaning
0-5	Hop Count	Number of path hashes (0 - 63)
6-7	Hash Size Code	Stored as hash_size - 1

Hash size codes:

Bits 6-7	Hash Size	Notes
0b00	1 byte	Legacy / default mode
0b01	2 bytes	Supported in current firmware
0b10	3 bytes	Supported in current firmware
0b11	4 bytes	Reserved / invalid

Examples:

- 0x00: zero-hop packet, no path bytes
- 0x05: 5 hops using 1-byte hashes, so path is 5 bytes

- 0x45: 5 hops using 2-byte hashes, so path is 10 bytes
- 0x8A: 10 hops using 3-byte hashes, so path is 30 bytes

Payload Types

Value	Name	Description
0x00	PAYLOAD_TYPE_REQ	Request (destination/source hashes + MAC)
0x01	PAYLOAD_TYPE_RESPONSE	Response to REQ or ANON_REQ
0x02	PAYLOAD_TYPE_TXT_MSG	Plain text message
0x03	PAYLOAD_TYPE_ACK	Acknowledgment
0x04	PAYLOAD_TYPE_ADVERT	Node advertisement
0x05	PAYLOAD_TYPE_GRP_TXT	Group text message (unverified)
0x06	PAYLOAD_TYPE_GRP_DATA	Group datagram (unverified)
0x07	PAYLOAD_TYPE_ANON_REQ	Anonymous request
0x08	PAYLOAD_TYPE_PATH	Returned path
0x09	PAYLOAD_TYPE_TRACE	Trace a path, collecting SNR for each hop
0x0A	PAYLOAD_TYPE_MULTIPART	Packet is part of a sequence of packets
0x0B	PAYLOAD_TYPE_CONTROL	Control packet data (unencrypted)
0x0C	reserved	reserved
0x0D	reserved	reserved
0x0E	reserved	reserved
0x0F	PAYLOAD_TYPE_RAW_CUSTOM	Custom packet (raw bytes, custom encryption)

Payload Versions

Value	Version	Description
0x00	1	1-byte src/dest hashes, 2-byte MAC
0x01	2	Future version (e.g., 2-byte hashes, 4-byte MAC)
0x02	3	Future version
0x03	4	Future version

Payload Format

Inside each [MeshCore Packet](#) is a payload, identified by the payload type in the packet header. The types of payloads are:

- Node advertisement.
- Acknowledgment.
- Returned path.
- Request (destination/source hashes + MAC).
- Response to REQ or ANON_REQ.
- Plain text message.
- Anonymous request.
- Group text message (unverified).
- Group datagram (unverified).
- Multi-part packet

- Control data packet
- Custom packet (raw bytes, custom encryption).

This document defines the structure of each of these payload types.

NOTE: all 16 and 32-bit integer fields are Little Endian.

Important concepts:

- Node hash: the first byte of the node's public key

Node advertisement

This kind of payload notifies receivers that a node exists, and gives information about the node

Field	Size (bytes)	Description
public key	32	Ed25519 public key of the node
timestamp	4	unix timestamp of advertisement
signature	64	Ed25519 signature of public key, timestamp, and app data
appdata	rest of payload	optional, see below

Appdata

Field	Size (bytes)	Description
flags	1	specifies which of the fields are present, see below
latitude	4 (optional)	decimal latitude multiplied by 1000000, integer
longitude	4 (optional)	decimal longitude multiplied by 1000000, integer
feature 1	2 (optional)	reserved for future use
feature 2	2 (optional)	reserved for future use
name	rest of appdata	name of the node

Appdata Flags

Value	Name	Description
0x01	is chat node	advert is for a chat node
0x02	is repeater	advert is for a repeater
0x03	is room server	advert is for a room server
0x04	is sensor	advert is for a sensor server
0x10	has location	appdata contains lat/long information
0x20	has feature 1	Reserved for future use.
0x40	has feature 2	Reserved for future use.
0x80	has name	appdata contains a node name

Acknowledgement

An acknowledgement that a message was received. Note that for returned path messages, an acknowledgement can be sent in the "extra" payload (see [Returned Path](#)) instead of as a separate acknowledgement packet. CLI commands do not cause acknowledgement responses, neither discrete nor extra.

Field	Size (bytes)	Description
checksum	4	CRC checksum of message timestamp, text, and sender pubkey

Returned path, request, response, and plain text message

Returned path, request, response, and plain text messages are all formatted in the same way. See the subsection for more details about the ciphertext's associated plaintext representation.

Field	Size (bytes)	Description
destination hash	1	first byte of destination node public key
source hash	1	first byte of source node public key
cipher MAC	2	MAC for encrypted data in next field
ciphertext	rest of payload	encrypted message, see subsections below for details

Returned path

Returned path messages provide a description of the route a packet took from the original author. Receivers will send returned path messages to the author of the original message.

Field	Size (bytes)	Description
path length	1	length of next field
path	see above	a list of node hashes (one byte each)
extra type	1	extra, bundled payload type, eg., acknowledgement or response. Same values as in Packet Format
extra	rest of data	extra, bundled payload content, follows same format as main content defined by this document

Request

Field	Size (bytes)	Description
timestamp	4	sender time (unix timestamp)
request data	rest of payload	application-defined request payload body

For the common chat/server helpers in BaseChatMesh, the current request type values are:

Value	Name	Description
0x01	get stats	get stats of repeater or room server
0x02	keepalive	keep-alive request used for maintained connections

Get stats

Gets information about the node, possibly including the following:

- Battery level (millivolts)
- Current transmit queue length
- Current free queue length
- Last RSSI value
- Number of received packets
- Number of sent packets
- Total airtime (seconds)
- Total uptime (seconds)
- Number of packets sent as flood

- Number of packets sent directly
- Number of packets received as flood
- Number of packets received directly
- Error flags
- Last SNR value
- Number of direct route duplicates
- Number of flood route duplicates
- Number posted (?)
- Number of post pushes (?)

Get telemetry data

Not defined in BaseChatMesh. Sensor- and application-specific request payloads may be implemented by higher-level firmware.

Get Telemetry

Not defined in BaseChatMesh.

Get Min/Max/Ave (Sensor nodes)

Not defined in BaseChatMesh.

Get Access List

Not defined in BaseChatMesh.

Get Neighbors

Not defined in BaseChatMesh.

Get Owner Info

Not defined in BaseChatMesh.

Response

Field	Size (bytes)	Description
content	rest of payload	application-defined response body

Response contents are opaque application data. There is no single generic response envelope beyond the encrypted payload wrapper shown above.

Plain text message

Field	Size (bytes)	Description
timestamp	4	send time (unix timestamp)
txt_type + attempt	1	upper six bits are txt_type (see below), lower two bits are attempt number (0..3)

Field	Size (bytes)	Description
message	rest of payload	the message content, see next table
txt_type		
Value	Description	Message content
0x00	plain text message	the plain text of the message
0x01	CLI command	the command text of the message
0x02	signed plain text message	first four bytes is sender pubkey prefix, followed by plain text message

Anonymous request

Field	Size (bytes)	Description
destination hash	1	first byte of destination node public key
public key	32	sender's Ed25519 public key
cipher MAC	2	MAC for encrypted data in next field
ciphertext	rest of payload	encrypted message, see below for details

Room server login

Field	Size (bytes)	Description
timestamp	4	sender time (unix timestamp)
sync timestamp	4	sender's "sync messages SINCE x" timestamp
password	rest of message	password for room

Repeater/Sensor login

Field	Size (bytes)	Description
timestamp	4	sender time (unix timestamp)
password	rest of message	password for repeater/sensor

Repeater - Regions request

Field	Size (bytes)	Description
timestamp	4	sender time (unix timestamp)
req type	1	0x01 (request sub type)
reply path len	1	path len for reply
reply path	(variable)	reply path

Repeater - Owner info request

Field	Size (bytes)	Description
timestamp	4	sender time (unix timestamp)
req type	1	0x02 (request sub type)
reply path len	1	path len for reply
reply path	(variable)	reply path

Repeater - Clock and status request

Field	Size (bytes)	Description
timestamp	4	sender time (unix timestamp)
req type	1	0x03 (request sub type)
reply path len	1	path len for reply
reply path	(variable)	reply path

Group text message

Field	Size (bytes)	Description
channel hash	1	first byte of SHA256 of channel's shared key
cipher MAC	2	MAC for encrypted data in next field
ciphertext	rest of payload	encrypted message, see below for details

The plaintext contained in the ciphertext matches the format described in [plain text message](#). Specifically, it consists of a four byte timestamp, a flags byte, and the message. The flags byte will generally be 0x00 because it is a "plain text message". The message will be of the form <sender name>: <message body> (eg., user123: I'm on my way).

The sender name is unverified message text. Group messages contain no sender signature, so any channel-key holder can choose any sender name.

Group datagram

Field	Size (bytes)	Description
channel hash	1	first byte of SHA256 of channel's shared key
cipher MAC	2	MAC for encrypted data in next field
ciphertext	rest of payload	encrypted data, see below for details

The data contained in the ciphertext uses the format below:

Field	Size (bytes)	Description
data type	2	Identifier for type of data. (See number_allocations.md)
data len	1	byte length of data
data	rest of payload	(depends on data type)

Control data

Field	Size (bytes)	Description
flags	1	upper 4 bits is sub_type
data	rest of payload	typically unencrypted data

DISCOVER_REQ (sub_type)

Field	Size (bytes)	Description
flags	1	0x8 (upper 4 bits), prefix_only (lowest bit)
type_filter	1	bit for each ADV_TYPE_*
tag	4	randomly generate by sender
since	4	(optional) epoch timestamp (0 by default)

DISCOVER_RESP (sub_type)

Field	Size (bytes)	Description
flags	1	0x9 (upper 4 bits), node_type (lower 4)
snr	1	signed, SNR*4
tag	4	reflected back from DISCOVER_REQ
pubkey	8 or 32	node's ID (or prefix)

Custom packet

Custom packets have no defined format.